Evidencias Makefile

Instituto Politécnico Nacional IPN

Escuela Superior de Computo ESCOM

Alumnos:

López Ramos Marcos Isaid

Unidad de aprendizaje: Internet de las cosas | Embedded systems

Grupo: 6CM3

Profesor: Aleman Arce Miguel Angel

Ejercicio del repositorio de ejemplo (Parte 1)

Entramos a la carpeta julia y limpiamos con Make clean

- Se utilizó el comando `make clean` para eliminar archivos generados anteriormente como archivos objeto, ejecutables, archivos `.txt` y `.png`, permitiendo iniciar la ejecución desde cero.

```
/c/Users/nonba/OneDrive/Documentos/IoT/Embedded-Systems-IoT/Makefiles/julia

nonba@MSI UCRT64 ~
$ cd /c/Users/nonba/OneDrive/Documentos/IoT/Embedded-Systems-IoT/Makefiles/julia

nonba@MSI UCRT64 /c/Users/nonba/OneDrive/Documentos/IoT/Embedded-Systems-IoT/Makefiles/julia
$ make clean
rm *.o julia *.txt *.png

nonba@MSI UCRT64 /c/Users/nonba/OneDrive/Documentos/IoT/Embedded-Systems-IoT/Makefiles/julia
```

Mostramos los elementos que tiene la carpeta antes de ejecutar

- Con ls se verificó la estructura inicial del proyecto y los archivos disponibles antes de ejecutar el Makefile.

```
nonba@MSI UCRT64 /c/Users/nonba/OneDrive/Documentos/IoT/Embedded-Systems-IoT/Makefiles/julia
$ ls
Makefile  Makefile.old  README.md  julia_set.gp  main.cpp  test.gp

nonba@MSI UCRT64 /c/Users/nonba/OneDrive/Documentos/IoT/Embedded-Systems-IoT/Makefiles/julia
$
```

Ejecución del Makefile completo

- Al ejecutar make, el Makefile realizó automáticamente la compilación del programa, la creación del ejecutable, su ejecución y la generación de la gráfica mediante gnuplot.

```
nonba@MSI UCRT64 /c/Users/nonba/OneDrive/Documentos/IoT/Embedded-Systems-IoT/Makefiles/julia
$ make
Compiling
g++ -std=c++23 -O3 -c main.cpp -o main.o
Linking
g++ -std=c++23 -O3 main.o -o julia
RUN
./julia
Plot
gnuplot julia_set.gp
PNG Mode Selected
couldn't load font "CMU Serif Not-Rotated 5700", falling back to "Sans Not-Rotated 5700", expect ugl
y output.Open
xdg-open julia_set.png &
/bin/sh: line 1: xdg-open: command not found

nonba@MSI UCRT64 /c/Users/nonba/OneDrive/Documentos/IoT/Embedded-Systems-IoT/Makefiles/julia
$
```

Vemos los archivos generados

- Después de la ejecución se generaron nuevos archivos como main.o, julia, julia_set.txt y julia_set.png, comprobando que el proceso se realizó correctamente.

```
nonba@MSI UCRT64 /c/Users/nonba/OneDrive/Documentos/IoT/Embedded-Systems-IoT/Makefiles/julia
$ ls
Makefile      README.md  julia_set.gp   julia_set.txt  main.o
Makefile.old  julia.exe  julia_set.png  main.cpp       test.gp

nonba@MSI UCRT64 /c/Users/nonba/OneDrive/Documentos/IoT/Embedded-Systems-IoT/Makefiles/julia
$
```

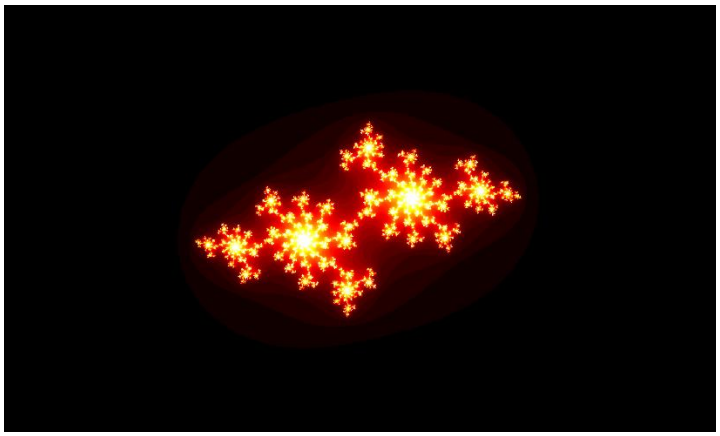
Mostar contenido del makefile con cat

- Se utilizó cat Makefile para visualizar el contenido del archivo y analizar cómo cada target contribuye a la automatización del proyecto.

```
nonba@MSI UCRT64 /c/Users/nonba/OneDrive/Documentos/IoT/Embedded-Systems-IoT/Makefiles/julia
$ cat makefile
# Variables
CXX = g++
CXXFLAGS = -std=c++23 -O3

GP = julia_set.gp
TXT = $(GP:.gp=.txt)
PNG = $(GP:.gp=.png)

SRCS = main.cpp

OBJS = $(SRCS:.cpp=.o)

APP  = julia

## TARGETS

all: run plot open

.PHONY: vars
vars:
	@echo "print variables"
	@echo "SRCS = $(SRCS)"
	@echo "OBJS = $(OBJS)"
	@echo "PRE  = $<"
	@echo "NAME = $@"
	@echo "GP   = $(GP)"
	@echo "TXT  = $(TXT)"

%.o: %.cpp
	@echo "Compiling"
	$(CXX) $(CXXFLAGS) -c $< -o $@

$(APP): $(OBJS)
	@echo "Linking"
	$(CXX) $(CXXFLAGS) $(OBJS) -o $(APP)

run: $(APP)
	@echo "RUN"
	./$(APP)

plot: $(TXT)
	@echo "Plot"
	gnuplot $(GP)

open:
	@echo "Open"
	xdg-open $(PNG) &

clean:
	rm *.o $(APP) *.txt *.png

nonba@MSI UCRT64 /c/Users/nonba/OneDrive/Documentos/IoT/Embedded-Systems-IoT/Makefiles/julia
$ |
```

Mostramos la imagen generada por Julia

- La imagen julia_set.png fue generada correctamente mediante gnuplot. Esto demuestra que el Makefile no solo compila el programa, sino que también automatiza la creación de resultados visuales.



Ejercicio proyecto propio (Parte 2)

Vemos la estructura del proyecto

- Con dir se comprobó la presencia de los archivos principales del proyecto: main.c, Makefile, README.md, .gitignore y la carpeta evidencia.

```
C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>dir
 El volumen de la unidad C es Windows
 El número de serie del volumen es: DADD-19AE

 Directorio de C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot

25/04/2026  12:30 p. m.    <DIR>          .
25/04/2026  11:52 a. m.    <DIR>          ..
25/04/2026  11:51 a. m.                42 .gitignore
25/04/2026  12:30 p. m.    <DIR>          evidencia
25/04/2026  11:50 a. m.               959 main.c
25/04/2026  11:51 a. m.               620 Makefile
25/04/2026  12:23 p. m.             4,199 README.md
               4 archivos          5,820 bytes
               3 dirs  203,509,563,392 bytes libres

C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>
```

Vemos la estructura del makefile con type makefile

- Se utilizó type Makefile para revisar el contenido del archivo y confirmar los targets principales como all, run, clean, rebuild y help.

```
C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>type Makefile
CC = gcc
CFLAGS = -Wall -Wextra -O2
TARGET = temperatura
SRCS = main.c
OBJS = $(SRCS:.c=.o)

.PHONY: all run clean rebuild help

all: $(TARGET)

help:
	@echo "Comandos disponibles:"
	@echo " mingw32-make        -> compila el programa"
	@echo " mingw32-make run    -> compila y ejecuta"
	@echo " mingw32-make clean  -> elimina archivos generados"
	@echo " mingw32-make rebuild -> limpia y recompila"

%.o: %.c
	$(CC) $(CFLAGS) -c $< -o $@

$(TARGET): $(OBJS)
	$(CC) $(CFLAGS) $(OBJS) -o $@

run: $(TARGET)
	./$(TARGET)

clean:
	del /Q $(OBJS) $(TARGET).exe 2>nul || exit 0

rebuild: clean all
C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>
```

Limpiamos archivos con mingw32-make clean

- Este comando eliminó automáticamente los archivos generados anteriormente como main.o y temperatura.exe, dejando el proyecto limpio para recompilar.

```
C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>mingw32-make clean
del /Q main.o temperatura.exe 2>nul || exit 0

C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>
```

Comprobamos la eliminación

- Con dir se verificó que los archivos temporales fueron eliminados correctamente y que solo permanecieron los archivos principales del proyecto.

```
C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>dir
 El volumen de la unidad C es Windows
 El número de serie del volumen es: DADD-19AE

 Directorio de C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot

25/04/2026  12:30 p. m.    <DIR>          .
25/04/2026  11:52 a. m.    <DIR>          ..
25/04/2026  11:51 a. m.                42 .gitignore
25/04/2026  12:30 p. m.    <DIR>          evidencia
25/04/2026  11:50 a. m.               959 main.c
25/04/2026  11:51 a. m.               620 Makefile
25/04/2026  12:23 p. m.             4,199 README.md
               4 archivos          5,820 bytes
               3 dirs  203,500,851,200 bytes libres

C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>
```

Compilación del makefile

- Al ejecutar mingw32-make, el sistema compiló main.c, generó main.o y posteriormente creó el ejecutable final temperatura.exe.

```
C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>mingw32-make
gcc -Wall -Wextra -O2 -c main.c -o main.o
gcc -Wall -Wextra -O2 main.o -o temperatura

C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>
```

Ejecución del programa

- Con mingw32-make run, el programa se ejecutó correctamente y permitió realizar la conversión de temperatura seleccionada por el usuario.

```
C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>mingw32-make run
gcc -Wall -Wextra -O2 main.o -o temperatura
./temperatura
=== Conversor de Temperatura IoT ===
1. Celsius a Fahrenheit
2. Fahrenheit a Celsius
3. Celsius a Kelvin
Seleccione una opcion: 1
Ingrese la temperatura: 32
Resultado: 89.60 °F

C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>
```

Aplicamos rebuild

- El comando mingw32-make rebuild primero limpió los archivos generados y después recompiló completamente el proyecto, demostrando una automatización más eficiente.

```
C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>mingw32-make rebuild
del /Q main.o temperatura.exe 2>nul || exit 0
gcc -Wall -Wextra -O2 -c main.c -o main.o
gcc -Wall -Wextra -O2 main.o -o temperatura

C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>
```

Vemos el directorio

- Finalmente se verificó el directorio del proyecto para confirmar la presencia del ejecutable temperatura.exe y los archivos necesarios para su correcto funcionamiento.

```
C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>dir
 El volumen de la unidad C es Windows
 El número de serie del volumen es: DADD-19AE

 Directorio de C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot

25/04/2026  01:11 p. m.    <DIR>          .
25/04/2026  11:52 a. m.    <DIR>          ..
25/04/2026  11:51 a. m.                42 .gitignore
25/04/2026  12:30 p. m.    <DIR>          evidencia
25/04/2026  11:50 a. m.               959 main.c
25/04/2026  01:11 p. m.             1,896 main.o
25/04/2026  11:51 a. m.               620 Makefile
25/04/2026  12:23 p. m.             4,199 README.md
25/04/2026  01:11 p. m.           140,622 temperatura.exe
               6 archivos        148,338 bytes
               3 dirs  203,496,001,536 bytes libres

C:\Users\nonba\OneDrive\Documentos\IoT\makefile-temperatura-iot>
```

Conclusión

Durante esta práctica demostré que el uso de Makefiles facilita la automatización de procesos, tanto en el ejemplo de Julia como en mi propio proyecto, (Conversor de Temperatura para IoT), también se observó que un Makefile permite reducir errores manuales, ahorrar tiempo y mantener una mejor organización del código.